DATA STRUCTURES & SYSTEMS DESIGN

Solution & Design Report

An O(1) LRU cache, a sweep-line event scheduler, and a deterministic room-allocation engine — with architecture diagrams, complexity analysis, concurrency strategy, and a verified JUnit 5 test suite.

PREPARED BY

Akshay Mehta

AI / Multi-Agent Systems Intern

Repository: github.com/Mrakshaymehta/avaloka-systems-assignment

Build: Java 17 · Maven · JUnit 5 | Status: 29/29 tests passing

LRU CACHE · EVENT SCHEDULER · ROOM ASSIGNMENT · CONCURRENCY

00

EXECUTIVE OVERVIEW

What was built and why it matters

Three systems problems, one consistent engineering approach.

This report documents the complete solution submitted in the repository. Each problem was treated as a small systems-design exercise: define the invariants first, choose the data structures that make those invariants cheap to maintain, then verify the behavior with focused unit tests. The result is three production-shaped components with predictable complexity and explicit trade-offs.

01 · LRU CACHE

O(1)

average get and put

HashMap for direct key access + custom doubly linked list with sentinels for recency and eviction.

02 · EVENT SCHEDULER

O(n log n)

conflict check and minimum rooms

Sort by start time, then scan or sweep with a min-heap of active end times.

03 · ROOM ASSIGNER

Peak-optimal

deterministic Room A, B, C…

Active-allocation heap plus a recycled-name queue guarantees no idle proliferation of rooms.

Design principles used throughout

- ✓ **Make the invariant explicit.** For the cache: map membership always equals list membership, and the node before `tail` is always the eviction candidate.
- ✓ **Prefer structure over scanning.** A hash map or heap replaces repeated linear scans when the operation is latency-critical.
- ✓ **Separate computation from presentation.** The scheduler decides *how many* rooms are needed; the assigner decides *which named room* is used.
- ✓ **Never mutate caller input.** Every scheduler operation works on a defensive copy.
- ✓ **Return immutable results.** `ScheduledEvent` is a read-only value object.
- ✓ **Make output deterministic.** Sort ties are broken explicitly, so identical inputs always produce identical room names.

VERIFICATION SUMMARY

29 JUnit 5 tests, 0 failures. The suite covers normal operations, boundary conditions, eviction ordering, adjacent meetings, empty input, room reuse, chronological output, naming beyond “Room Z”, and input immutability. Build command: `mvn clean test` .

NAVIGATION

Contents

01 **LRU Cache — design, architecture, and correctness** — O(1) operations, sentinel list, eviction walkthrough

02 **Event Scheduler — conflicts and minimum rooms** — Half-open intervals, sorting, sweep-line min-heap

03 **Room Assigner — names, reuse, and determinism** — Active allocation heap + recycling queue

04 **Testing and verification** — 29 tests, boundary coverage, build output

05 **Final discussion and analysis** — Complexity, trade-offs, future proofing, concurrency

Submission at a glance

SOURCE

4 classes

cache · scheduler · assigner · result

VERIFICATION

29 / 29

JUnit 5 tests passing

BUILD

Java 17

Maven · `mvn clean test`

Repository map

PATH	RESPONSIBILITY
`LRUCache.java`	O(1) get/put, sentinel doubly linked list, eviction.
`EventScheduler.java`	Conflict detection and minimum room count.
`RoomAssigner.java`	Deterministic named-room allocation and recycling.
`ScheduledEvent.java`	Immutable result object: start, end, roomId.
`src/test/…`	`LRUCacheTest` and `EventSchedulerTest` — 29 tests.

How to evaluate

1. Review the architecture and invariants in sections 01–03.
2. Run `mvn clean test` to reproduce the 29 passing tests.
3. Check the trade-off and concurrency reasoning in section 05.

```
$ mvn clean test
[INFO] Tests run: 29
[INFO] BUILD SUCCESS
```

01

COMPONENT ONE

LRU Cache

A fixed-capacity cache with O(1) reads, writes, and eviction.

The cache must answer two questions at the same time: *where is the value for this key?* and *which entry has not been used for the longest time?* A hash map answers the first in O(1) but has no ordering; a linked list answers the second in O(1) but cannot find a key without scanning. Combining them gives both properties.

Architecture

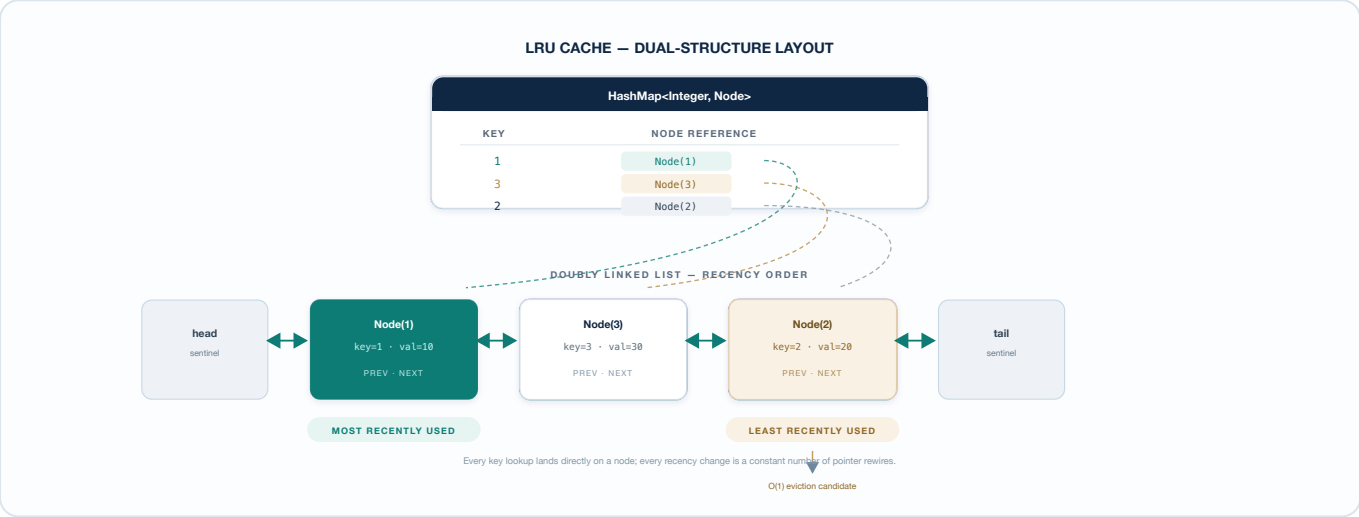


Figure 1. The hash map owns key lookup; the doubly linked list owns recency. The dummy `head` and `tail` remove null checks, and the node before `tail` is always the next eviction.

CORRECTNESS INVARIANTS

1. The map and list contain exactly the same nodes. **2.** `head.next` is the MRU entry; `tail.prev` is the LRU entry. **3.** A hit never changes cache size. **4.** A new key that exceeds capacity evicts exactly one node and removes its map entry. **5.** Updating an existing key never grows the cache.

Complexity

OPERATION	TIME	SPACE	NOTES
`get(key)`	`O(1) average`	`O(1)`	HashMap lookup plus constant pointer rewiring.
`put(key, value)`	`O(1) average`	`O(1)`	Insert or update, with at most one eviction.
Whole cache	—	`O(C)`	C = capacity; one map entry and one node per entry.

Worst-case HashMap operations are O(n) under pathological hash collisions, which is why the implementation is described as O(1) average.

Core implementation

The public surface is deliberately minimal. Ordering is delegated to four constant-time pointer helpers, so the cache logic stays readable and each invariant has a single owner.

LRUCACHE.JAVA — PUBLIC OPERATIONS

```
public int get(int key) {
    Node node = cache.get(key);
    if (node == null) {
        return -1;
    }
    moveToFront(node);          // recency update is a write
    return node.value;
}

public void put(int key, int value) {
    Node node = cache.get(key);
    if (node != null) {
        node.value = value;
        moveToFront(node);
        return;
    }

    Node newNode = new Node(key, value);
    cache.put(key, newNode);
    addNodeToFront(newNode);

    if (cache.size() > capacity) {
        Node lru = popTail();
        cache.remove(lru.key);  // node stores its key for this
    }
}
```

Helper methods

- `addNodeToFront(node)` — wire immediately after `head`.
- `removeNode(node)` — unlink and clear both pointers.
- `moveToFront(node)` — remove, then add to front.
- `popTail()` — remove `tail.prev`, the LRU node.

Why both structures?

- Hash map alone cannot identify the least recently used entry.
- Linked list alone needs O(n) to find a key.
- Together: lookup, promotion, insertion, and eviction are all O(1) average.

Walkthrough — capacity 2

OPERATION	LIST AFTER OPERATION (FRONT → BACK)	RESULT / REASON
`put(1, 10)`	`[1]`	Cache not full.
`put(2, 20)`	`[2, 1]`	2 is MRU; 1 is LRU.
`get(1)`	`[1, 2]`	Hit returns 10 and promotes 1.
`put(3, 30)`	`[3, 1]`	Full: evict tail (2), insert 3.
`get(2)`	`[3, 1]`	Miss returns −1; no recency change.
`put(1, 99)`	`[1, 3]`	Existing key: update value, promote.

02

COMPONENT TWO

Event Scheduler

Conflict detection and peak concurrency using sorting and a sweep line.

All scheduling logic uses the same interval model: each event is `{start, end}` and intervals are **half-open**. An event ending at time `t` and another starting at `t` are compatible. This single decision makes back-to-back meetings work naturally and is the difference between a correct scheduler and one that blocks a meeting room for an unnecessary minute.

INPUT

`{start, end}`

Half-open interval; end is exclusive.

CONFLICT

`currStart < prevEnd`

Strict inequality permits adjacency.

ROOM RELEASE

`end <= nextStart`

A room is reusable at its end time.

Conflict detection — `canAttendAll`

Sort a defensive copy of the events by start time, then compare each event with its immediate predecessor. Because the list is sorted, a previous event with an end strictly greater than the current start means the intervals overlap. If every pair is compatible, all events are compatible.

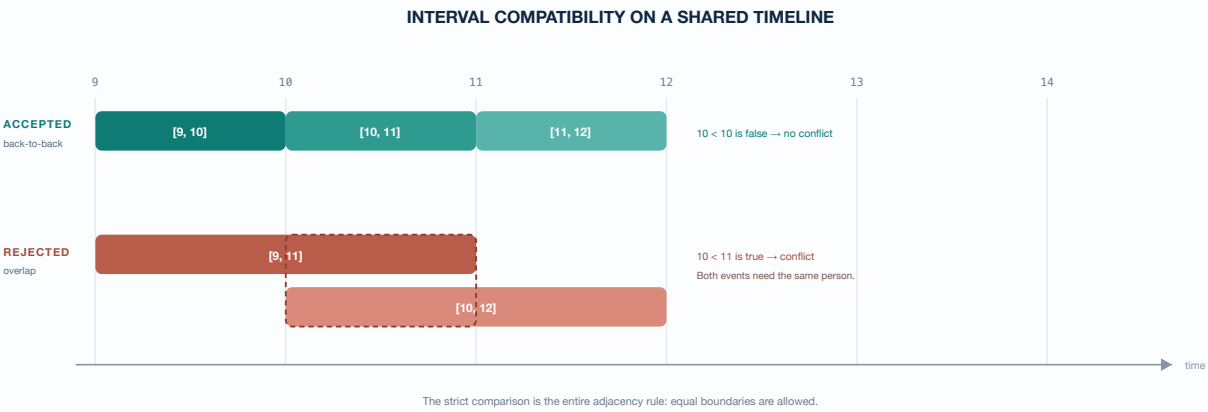


Figure 2. Adjacent intervals are compatible because the next start equals the previous end, not less than it. Any true overlap has a start strictly before the previous end.

EVENTSCHEDULER.JAVA — CONFLICT SCAN AFTER SORTING

```
for (int i = 1; i < sorted.length; i++) {
    int previousEnd = sorted[i - 1][1];
    int currentStart = sorted[i][0];
    if (currentStart < previousEnd) {
        return false;
    }
}
return true;
```

Minimum rooms — `minRoomsRequired`

This is a sweep-line problem. Sort by start time and maintain a min-heap of end times for every room currently in use. Before each event, if the earliest ending room is free (`heap.peek() <= start`), reuse it. Otherwise a new room is required. The number of rooms allocated by this greedy process is the peak concurrency.



Figure 3. Each event either reuses the earliest-released room or opens a new one. The heap top is the only room that can matter, making the decision O(log n) per event.

EVENTSCHEDULER.JAVA — SWEEP-LINE CORE

```
PriorityQueue<Integer> activeEndTimes = new PriorityQueue<>();

for (int[] event : sorted) {
    int start = event[0];
    int end = event[1];

    if (!activeEndTimes.isEmpty() && activeEndTimes.peek() <= start) {
        activeEndTimes.poll();       // earliest room is free; reuse it
    }
    activeEndTimes.offer(end);       // occupy a room until end
}
return activeEndTimes.size();
```

Worked example

EVENT	HEAP BEFORE	DECISION	HEAP AFTER	ROOMS
{0, 30}	[]	Open a room.	[30]	1
{5, 10}	[30]	30 > 5, earliest room still busy.	[10, 30]	2
{15, 20}	[10, 30]	10 ≤ 15, release and reuse that room.	[20, 30]	2

Why greedy is optimal. Peak overlap is a lower bound on rooms. The sweep opens a room only when every room is busy, so its peak allocation equals that lower bound.

03

COMPONENT THREE

Room Assigner

Turning a room count into stable, human-readable room names.

Counting rooms is only half of the problem. The assigner must decide which named room hosts each event, reuse names once a room is free, and return a clear data structure that a UI, calendar, or downstream service can consume. The implementation uses an active-allocation heap ordered by end time and a recycling queue ordered by room name.

Allocation pipeline

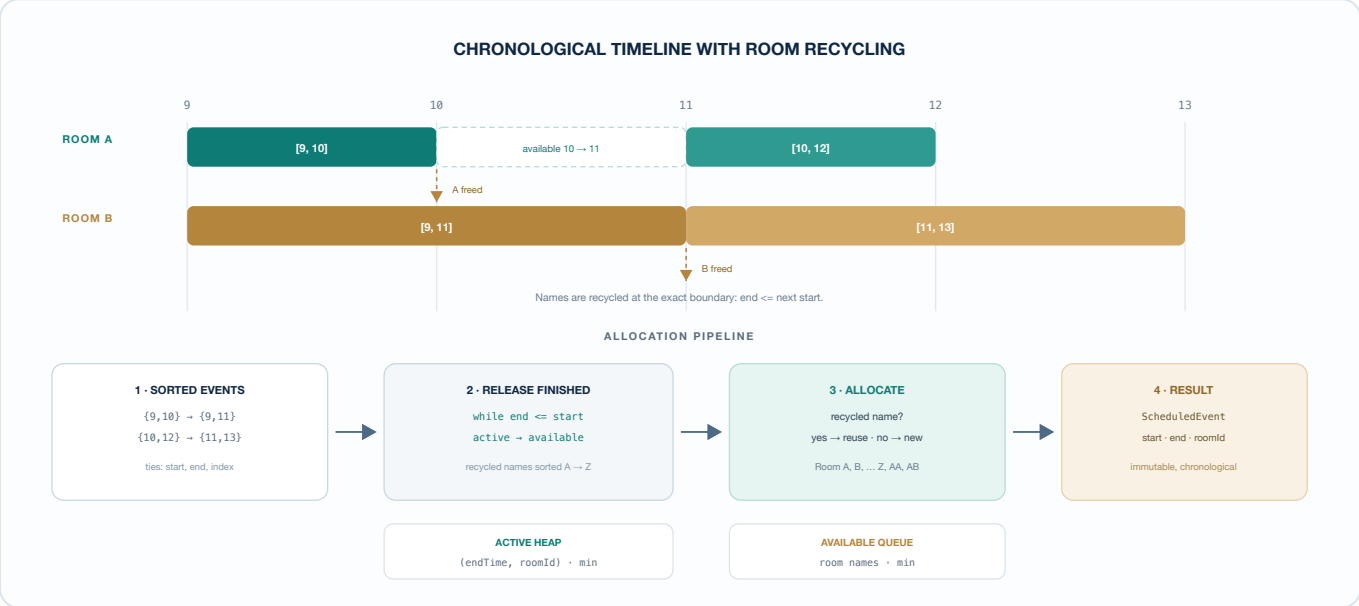


Figure 4. Finished allocations move into the recycling queue. A new room name is generated only when no recycled name is available, so distinct names equal peak concurrency.

ROOMASSIGNER.JAVA — RELEASE, RECYCLE, ALLOCATE

```
for (EventSlot event : ordered) {
    releaseFinishedRooms(active, availableRooms, event.start());

    String roomId = availableRooms.poll();
    if (roomId == null) {
        roomId = roomName(nextRoomNumber++);
    }

    active.offer(new ActiveAllocation(event.end(), roomId));
    scheduled.add(new ScheduledEvent(event.start(), event.end(), roomId));
}
```

EVENT	FREE AT START?	ASSIGNED	WHY
`{9, 10}`	—	ROOM A	First allocation; no recycled names.
`{9, 11}`	No	ROOM B	Room A is occupied until 10.
`{10, 12}`	Yes, A	ROOM A	A ends at 10, so it is free at the boundary.
`{11, 13}`	Yes, B	ROOM B	B ends at 11, so it is free at the boundary.

State evolution

AFTER EVENT	ACTIVE HEAP (END, ROOM)	RECYCLED NAMES	DECISION
`{9, 10}`	`[(10, A)]`	`[]`	Open Room A.
`{9, 11}`	`[(10, A), (11, B)]`	`[]`	A busy → open Room B.
`{10, 12}`	`[(11, B), (12, A)]`	`[]`	A ends at 10 → reuse A.
`{11, 13}`	`[(12, A), (13, B)]`	`[]`	B ends at 11 → reuse B.

NAMING BEYOND Z

Room index → label uses base-26 spreadsheet labeling: 0 → A, 25 → Z, 26 → AA, 27 → AB. The namespace is effectively unbounded for practical schedules.

DETERMINISM

Events are ordered by start, then end, then original index. The available queue orders names alphabetically. Identical input therefore produces identical output every run.

OPTIMALITY GUARANTEE

Because names are reused at the exact release boundary and new names are generated only when every active room is busy, the number of distinct names equals the peak concurrency computed by `minRoomsRequired`. This is the minimum possible, since the peak is a lower bound.

Complexity

FUNCTION	TIME	SPACE	NOTES
`assignRooms`	$O(n \log n)$	$O(n)$	Sorting + one heap operation per event.
`releaseFinishedRooms`	$O(\log n)$ per release	$O(1)$	Each room moves between queues at most once.
`roomName`	$O(\log_{26} k)$	$O(\log_{26} k)$	Base-26 label generation.

04

VERIFICATION

Testing and Results

Boundary-first tests, not just happy paths.

The suite is organized around behavior that is easy to get wrong: LRU ordering after a read, updating an existing key without growing the cache, strict inequality at interval boundaries, empty input, deterministic ordering, and room-name recycling. Every test is independent and uses only the public API.

TEST CLASS	TESTS	COVERAGE FOCUS
`LRUCacheTest`	9	Basic get/put, eviction order, MRU promotion on read and update, missing keys, capacity one, repeated updates, invalid capacity.
`EventSchedulerTest` · `CanAttendAll`	6	Overlap, containment, non-overlap, back-to-back adjacency, unsorted input, empty and single-event input.
`EventSchedulerTest` · `MinRoomsRequired`	7	Empty input, single event, reuse of a room for adjacent events, peak concurrency, three-way overlap, unsorted input, input immutability.
`EventSchedulerTest` · `RoomAssignment`	7	Sequential reuse, overlapping allocation, Rooms A/B/C, chronological output, empty input, naming past Z, input immutability.
Total	**29**	**0 failures · 0 errors · 0 skipped**

Representative assertions

LRU EVICTION AND PROMOTION

```
cache.put(1, 1);
cache.put(2, 2);
cache.get(1);       // 1 becomes MRU
cache.put(3, 3);    // evicts 2

assertEquals(1, cache.get(1));
assertEquals(-1, cache.get(2));
assertEquals(3, cache.get(3));
```

ADJACENCY AND ROOM REUSE

```
// 10 == 10, so this is allowed
assertTrue(canAttendAll(
    new int[][]{{9,10}, {10,11}}));

// A ends at 10, so it is reused
assertEquals(List.of(
    new ScheduledEvent(9, 10, "Room A"),
    new ScheduledEvent(10, 12, "Room A")
), RoomAssigner.assignRooms(
    new int[][]{{9,10}, {10,12}}));
```

Automated build output

```
$ mvn clean test
[INFO] Running com.avaloka.assignment.EventSchedulerTest$RoomAssignment
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.avaloka.assignment.EventSchedulerTest$MinRoomsRequired
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.avaloka.assignment.EventSchedulerTest$CanAttendAll
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.avaloka.assignment.LRUCacheTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0

[INFO] Results:
[INFO] Tests run: 29, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Why these tests matter

ORDERING BUGS

The most common LRU defect is evicting in insertion order instead of recency order. Promotion-on-read and promotion-on-update tests catch it.

BOUNDARY BUGS

Using `<=` instead of `<` would reject valid back-to-back meetings. Dedicated adjacency tests pin the convention.

NAMING BUGS

Room reuse can silently skip or duplicate names. Exact `ScheduledEvent` equality checks verify both identity and sequence.

Boundary-case matrix

INPUT	EXPECTED BEHAVIOR	GUARDED BY
Empty or null event array	No conflict, 0 rooms, empty schedule.	All scheduler methods.
Adjacent `{9,10}`, `{10,11}`	Compatible; one room reused.	Strict `<` and release `<=`.
Three-way overlap	Three rooms; names A, B, C.	Peak concurrency tests.
More than 26 concurrent rooms	Names continue at AA, AB, …	`roomName` tests.
Unsorted input	Same result as sorted input; caller array untouched.	Defensive copy tests.

Test design strategy

- One behavior per test; no shared mutable fixtures.
- Assertions use exact values and deterministic ordering.
- Boundary conditions are tested explicitly, not inferred.

REPRODUCE A SINGLE SUITE

```
mvn -Dtest=LRUCacheTest test
mvn -Dtest=EventSchedulerTest test
```

Useful while iterating; `mvn clean test` runs the full build.

05

FINAL DISCUSSION & ANALYSIS

Complexity, trade-offs, evolution, concurrency

The engineering reasoning behind the implementation choices.

1. Time and space complexity

Notation: n = number of events, C = cache capacity, k = number of distinct room names. Hash-based bounds are average case.

FUNCTION	TIME	SPACE	REASONING
`LRUCache(int capacity)`	`O(C)`	`O(C)`	Map capacity derived from C; sentinels are constant.
`get(int key)`	`O(1) avg`	`O(1)`	Map lookup + constant pointer rewiring.
`put(int, int)`	`O(1) avg`	`O(1)`	Update, insert, and at most one eviction.
`size()`	`O(1)`	`O(1)`	Delegates to `HashMap.size()`.
`canAttendAll`	`O(n log n)`	`O(n)`	Defensive copy + sort; O(n) scan.
`minRoomsRequired`	`O(n log n)`	`O(n)`	Sort + one heap push/pop per event.
`assignRooms`	`O(n log n)`	`O(n)`	Sort + active heap + recycling queue + result.
`roomName(k)`	$O(\log_{26} k)$	$O(\log_{26} k)$	Label length grows logarithmically.

Sorting dominates the scheduling components. Even if the input is already sorted, the public contract does not require it, so the implementation sorts defensively. If profiling showed that callers always provide sorted data, a separate API could accept a sorted precondition; that would reduce the scan to O(n), but it would make the method easier to misuse.

2. Trade-offs: why HashMap + doubly linked list is optimal

STRUCTURE	LOOKUP	PROMOTE	EVICT	VERDICT
HashMap only	O(1)	—	—	No recency information.
Array / ArrayList	O(n)	O(n)	O(n)	Shifting dominates.
Singly linked list	O(n)	O(n)	O(n)	Cannot unlink without predecessor.
Balanced tree	O(log n)	O(log n)	O(log n)	Correct but unnecessarily slow.
HashMap + doubly linked list	**O(1)**	**O(1)**	**O(1)**	**Optimal for all four operations.**

The map provides direct access; the doubly linked list provides bidirectional splicing in constant time. Sentinel nodes remove the empty-list and boundary special cases that are a common source of bugs. The cost is memory: each entry carries a key, a value, a map entry, and two references. That is the deliberate space-for-time trade-off at the heart of every LRU cache.

> **WHY NOT JUST USE LINKEDHASHMAP?**
>
> `LinkedHashMap` with `accessOrder = true` implements the same dual structure internally and is a reasonable production choice. The custom implementation is used here to make the mechanics explicit, to control eviction and naming behavior, and to provide a clear substrate for the concurrency discussion in section 5.4.

3. Future proofing: named room assignment

The room layer is deliberately separated from the scheduling math. `EventScheduler` computes how many rooms are needed; `RoomAssigner` decides which named room each event receives. That boundary is where the system can evolve without changing the scheduling core.

STABLE NAMING TODAY

- Deterministic labels: A, B, C … Z, AA, AB.
- Freed names are recycled through a min-heap.
- Distinct names = peak concurrency = optimal.
- Tie-breaking is explicit and reproducible.

PRODUCTION EXTENSIONS

- Room registry with stable external IDs.
- Capacity, floor, building, equipment, accessibility.
- Calendars, maintenance windows, time zones.
- Selection strategies: first-fit, best-fit, affinity.

A production `Room` would carry a stable external identifier separate from its display label, so renaming a room never breaks historical records. A strategy interface can choose among available rooms instead of always taking the alphabetically first name:

```
interface RoomSelectionStrategy {
    Optional<Room> select(Event event, List<Room> availableRooms);
}
```

If events also carry attendance requirements and rooms have capacities, the problem becomes a bin-packing variant and is NP-hard in general. Practical systems use greedy heuristics with conflict repair rather than exact optimization. Persisting the external room ID, not only the label, keeps historical assignments valid through renames and floor changes.

4. Concurrency: making the cache thread-safe

The submitted `LRUCache` is intentionally single-threaded. Its `get` is logically a read but physically a write, because it promotes the accessed node. Any concurrency strategy must treat both `get` and `put` as mutations.

Strategy A — coarse-grained ReentrantLock

One lock around every public operation is the simplest correct design. It supports `tryLock(timeout)`, fairness, and condition variables. The cost is serialized throughput and a single contention point.

```java
private final ReentrantLock lock = new ReentrantLock();

public int get(int key) {
    lock.lock();
    try {
        // map lookup, moveToFront, return value
    } finally {
        lock.unlock();
    }
}
```

Strategy B — why a ReadWriteLock does not help

A read-write lock only helps when readers do not mutate shared state. In an LRU cache, a `get` updates the recency list. Taking a write lock for reads gives no advantage over a plain mutex; taking a read lock would allow concurrent readers to corrupt the list. The only way to benefit is to relax recency: batch promotion updates, use a CLOCK approximation with reference bits, or maintain hit statistics outside the critical section. Without such a redesign, `ReentrantReadWriteLock` adds overhead without parallelism.

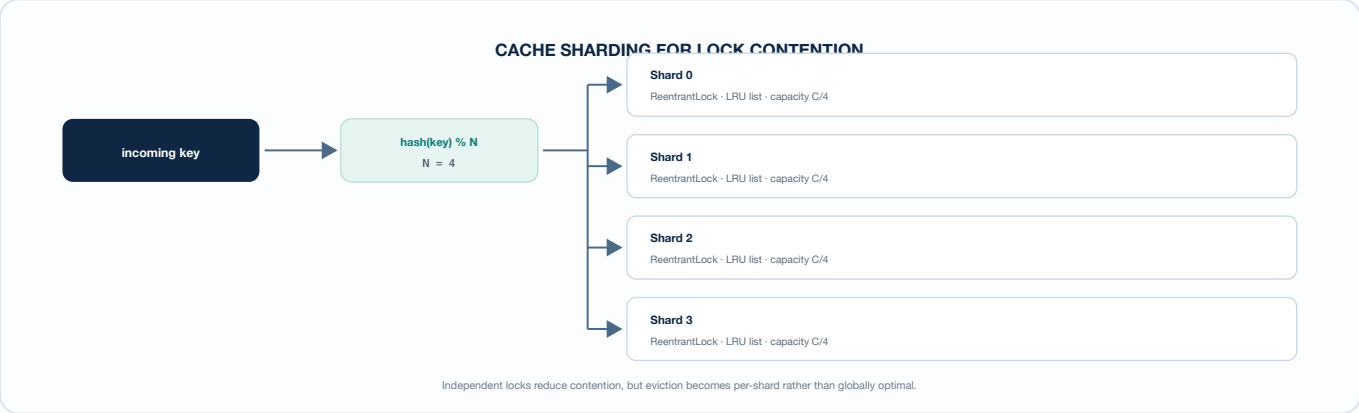


Figure 5. Sharding partitions keys across independent cache instances. Throughput improves, at the cost of an approximate global LRU policy and possible capacity skew.

Strategy C — cache sharding

Split the cache into `N` independent shards, each with its own map, list, and lock. Route by `hash(key) % N`. Contention drops by roughly a factor of `N`, but eviction becomes per-shard: a globally hot entry can be evicted from a cold shard. Hash imbalance can also fill one shard earlier than another. Virtual shards, rebalancing, or a global admission policy mitigate this. Cross-key operations such as global size or snapshots require coordinating every shard, and a single hot key still serializes on one lock.

Strategy D — lock-free or fine-grained

A `ConcurrentHashMap` plus an atomic linked list is possible but notoriously difficult: promotion touches multiple pointers that must change atomically, and ABA problems require versioned references. Production caches often serialize updates through a single writer, stripe locks over buckets, or use probabilistic admission and frequency sketches (for example TinyLFU in Caffeine) to decouple recency maintenance from the read path.

CORRECTNESS REQUIREMENTS FOR ANY CONCURRENT DESIGN

Map and list updates must be atomic with respect to each other; eviction must not remove a node just promoted by another thread; node fields need safe publication under the protecting lock or through volatile/atomic access; and `size()` must share the same synchronization domain if it participates in eviction decisions.

Conclusion

The submission delivers an O(1) LRU cache, an O(n log n) scheduler with a provably optimal room count, and a deterministic room assigner whose distinct names equal peak concurrency. The design keeps invariants explicit, avoids input mutation, and validates behavior across 29 tests. For a production deployment, the next steps would be a sharded or lock-protected cache wrapper, property-based scheduling tests, and a room registry with external IDs and capacity-aware selection.

BUILD

`mvn clean test`

Java 17 · Maven · JUnit 5.10.2

VERIFY

`29 / 29 passing`

0 failures · 0 errors · 0 skipped

REPOSITORY

`github.com/Mrakshaymehta/avaloka-systems-assignment`

Key takeaways

INVARIANTS FIRST

Choose structures that make the core invariant cheap to maintain, then let complexity fall out of the design.

BOUNDARIES ARE CONTRACT

Half-open intervals and strict comparisons are product decisions, not implementation details. Test them directly.

CONCURRENCY IS DESIGN

A thread-safe cache needs more than a lock around methods when reads mutate recency. Plan for that early.